RepoGuard

Secure commits before they reach your repository platform

Application Java desktop dashboard and CLI	**Build** Maven, Java 17 target	**Publication** GitHub main branch
Commit protection Pre-commit hook installers	**Automation** Jenkins SCM polling every 5 minutes	**Deployment** Ansible + systemd on Linux

RepoGuard scans local source and configuration files for common credential patterns. Developers can use the Swing dashboard, run a fail-closed command-line scan, or install a Git pre-commit hook. Maven packaging, Jenkins CI, and Ansible deployment assets are included.

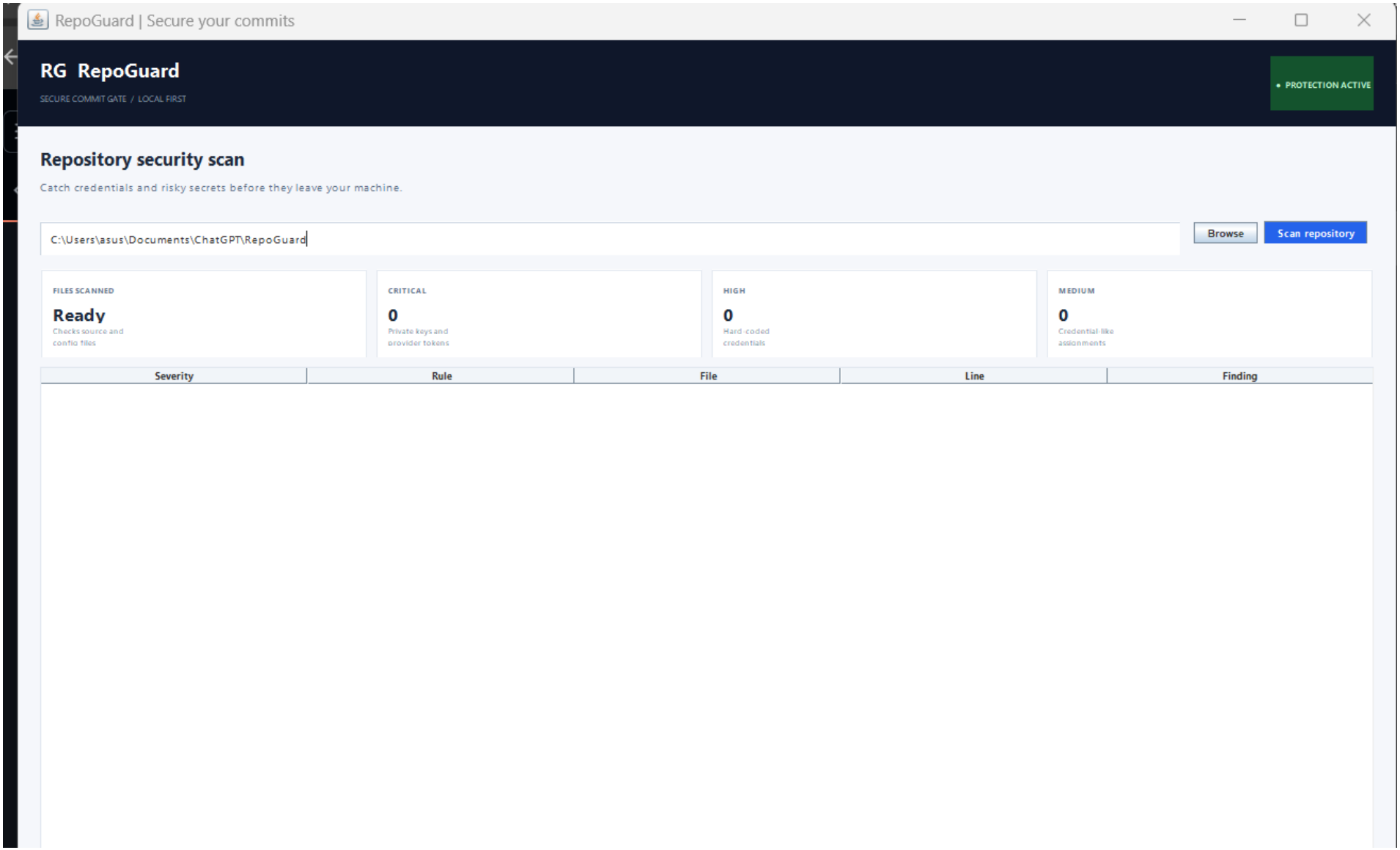
Verification captured: Maven clean verify passed all 3 unit tests; the full project scan returned PASS with no findings. RepoGuard is published on GitHub. Jenkins was already installed locally and is now configured with a passing RepoGuard pipeline; Ansible and a Linux deployment host were not available on this Windows workstation.

Repository: https://github.com/harisankarMallika/RepoGuard

Initial published commit: 8cc11889b7931e997168b4b035f91f07067a3290

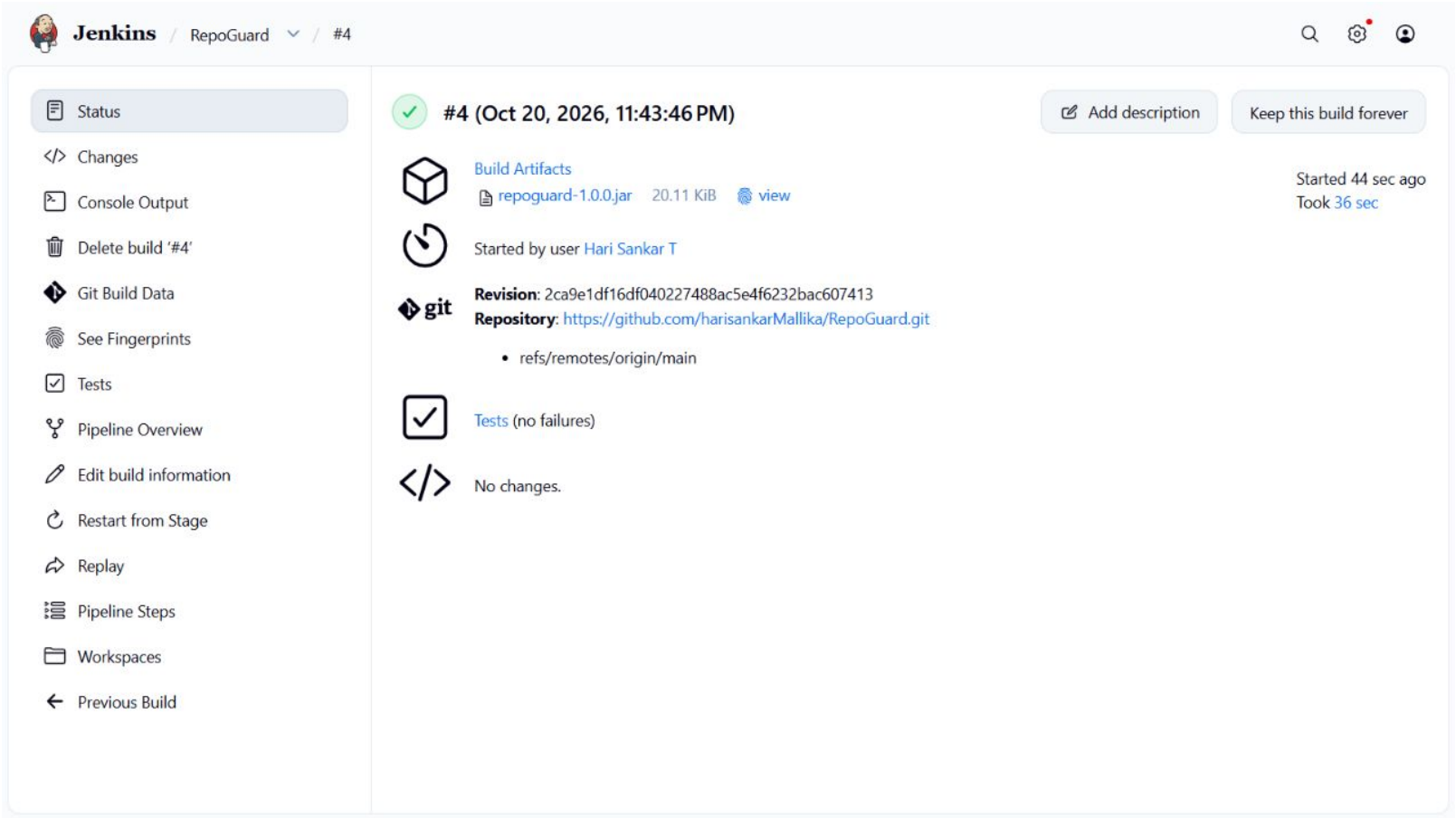
Application UI

Repository security dashboard captured from the running Java Swing application. Select a repository folder and scan to populate the file and severity counts and findings table.



Jenkins build status

Jenkins Pipeline build #4 completed successfully on commit 2ca9e1d. The run passed Maven verification, found no test failures or scanner findings, and archived the executable JAR.



Maven conversion and validation

The project uses Maven's standard source layout. `pom.xml` declares `io.repoguard:repoguard:1.0.0`, compiles with Java release 17, runs JUnit 5 tests, and produces an executable JAR with `com.repoguard.Main` as its entry point.

Build and run commands

```
mvn clean verify
java -jar target/repoguard-1.0.0.jar
java -jar target/repoguard-1.0.0.jar --scan C:\path\to\repository
```

The build passed on Java 26 while targeting Java 17. Maven reported 3 tests, 0 failures, 0 errors and produced `target/repoguard-1.0.0.jar`. The local CLI scanned 17 eligible project files and returned **PASS: no findings**; Jenkins scanned its 16-file checkout and also returned PASS. The scanner skips .git, target, common build and dependency directories, binary formats, and files larger than 1 MB.

Exit codes: 0 = clean scan; 1 = findings; 2 = usage or scan error. Findings report severity, relative file, line, and rule without printing the matched credential.

Built-in rules

AWS access key IDs, private-key headers, GitHub and Slack token formats, likely hard-coded credentials, and selected credential-style environment assignments are covered. These heuristics can miss unknown formats or flag false positives; they do not replace secret management or broader security analysis.

Git and GitHub commands

The local repository uses `main` and the requested HTTPS remote. The initial project commit and subsequent Jenkins configuration commit were pushed to GitHub; the current branch is main.

```
git remote add origin https://github.com/harisankarMallika/RepoGuard.git
git branch -M main
git add .
git status --short
git commit -m "Build RepoGuard secure commit scanner"
git push -u origin main
```

Status and verification commands

```
git status --short --branch
git remote -v
git log -1 --oneline
git ls-remote --heads origin
mvn -version
mvn clean verify
java -jar target/repoguard-1.0.0.jar --scan .
```

Verified publication: branch `main` tracking `origin/main`; initial commit `8cc1188` and Jenkins configuration commit `2ca9e1d` are on GitHub. This guide is included in a follow-up commit.

A Windows pre-commit hook is installed in this working copy. It runs the packaged CLI on the repository root and stops a commit when findings exist. Installers are in `scripts/install-hook.ps1` and `scripts/install-hook.sh`.

Jenkins configuration

The root `Jenkinsfile` defines a GitHub push trigger and five-minute SCM polling (`H/5 * * * *`). The job is configured to poll; because this Jenkins controller is local-only, no public GitHub webhook is configured and pushes are detected by polling within about five minutes.

Jenkins setup

1. Jenkins 2.578 is installed as a Windows service at `http://localhost:8080/`; Git, Pipeline, GitHub, JUnit, and Maven support are installed. Available plugin updates, including security fixes, were applied and the service restarted.
2. Jenkins Tools defines JDK `Java25` at `C:\Program Files\Java\jdk-25.0.4.1` and Maven tool `Maven` with installer version 3.10.0.
3. The `RepoGuard` Pipeline job uses SCM: GitHub HTTPS repository, branch `*/main`, script path `Jenkinsfile`, lightweight checkout; no credentials are needed for this public repository.
4. Both GitHub hook trigger and Poll SCM are enabled; the schedule is `H/5 * * * *`. No public webhook is configured for the localhost controller. To trigger immediately, deploy Jenkins at a secured public HTTPS address and add a GitHub push webhook to `/github-webhook/`.
5. The last successful build, #4, ran on commit `2ca9e1d`, passed all 3 tests, archived `target/repoguard-1.0.0.jar`, and reported no scanner findings.

Pipeline actions

```
Checkout SCM
mvn -B clean verify
archiveArtifacts artifacts: 'target/repoguard-*.jar'
Publish target/surefire-reports/*.xml
CLI smoke scan of workspace
```

The pipeline archives the JAR and publishes JUnit results. Credential findings make the build unstable; scanner execution errors fail the build. Build #1 exposed a Java tool alias mismatch and failed; after configuring the required Java25 alias and pushing the matching Jenkinsfile, builds #2, #3 and #4 succeeded.

Ansible deployment

The playbook `ansible/deploy.yml` targets Linux hosts in inventory group `repoguard`. It selects a Java 17 runtime package for Debian or Red Hat family hosts, copies the CLI-capable JAR to `/opt/repoguard`, renders a systemd oneshot unit, and enables and starts that service at boot.

Deployment commands

```
mvn clean package
cp ansible/inventory.ini.example ansible/inventory.ini
# Edit inventory.ini: hosts, SSH user, and repository path
ansible-playbook -i ansible/inventory.ini ansible/deploy.yml \
  -e repo_guard_jar="$(pwd)/target/repoguard-1.0.0.jar"
ansible -i ansible/inventory.ini repoguard -b -m command \
  -a 'systemctl status repoguard.service'
ansible -i ansible/inventory.ini repoguard -b -m command \
  -a 'journalctl -u repoguard.service -n 50 --no-pager'
```

Set `repo_guard_user` and `repo_guard_repository` in private inventory or group variables. The sample inventory uses a documentation-only IP address and must be replaced. The service scans once at start; add a systemd timer or Jenkins schedule for recurring remote scans.

Ansible was not installed on this Windows workstation and no Linux deployment host was supplied. The playbook is included but has not been executed against a remote host.

Project map

`src/main/java` Java application and scanner | `src/test/java` tests | `pom.xml` Maven build | `Jenkinsfile` CI | `ansible/` playbook and service template | `scripts/` Git hook installers | `docs/` operations notes

Ansible playbook excerpt

```
- name: Deploy RepoGuard scanner
  hosts: repoguard
  become: true
  vars:
    repo_guard_java_package: Debian or Red Hat Java 17 runtime
  tasks:
    - install Java 17 with the package module
    - create /opt/repoguard owned by repo_guard_user
    - copy the executable JAR to /opt/repoguard/repoguard.jar
    - render templates/repoguard.service.j2
    - enable and start repoguard.service
```